The `after_transaction` connection release mode is more efficient than the default JTA `after_statement` strategy, and so it should be used if the JTA transaction resource management logic does not interfere with this connection releasing strategy.

8.3 Monitoring connections

As previously concluded, using an externally configured `DataSource` is preferred because the actual `DataSource` can be decorated with connection pooling, monitoring and logging capabilities. Because that is exactly how FlexyPool[8] works too, the following diagram captures the `DataSource` proxying mechanism:

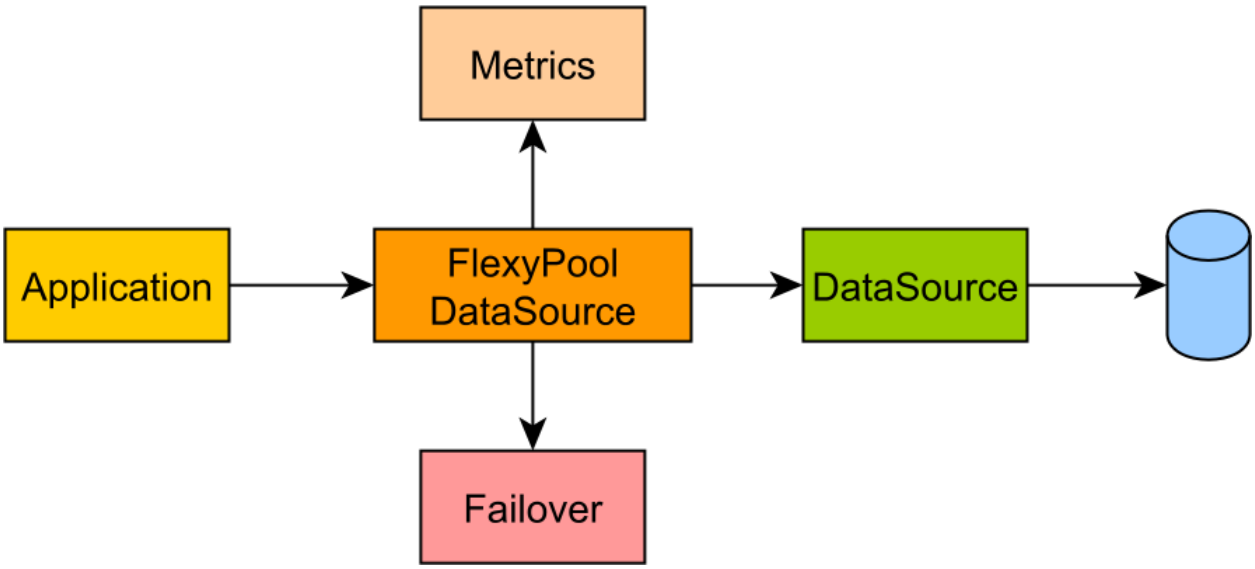

Figure 8.2: FlexyPool architecture

Instead of getting the actual `DataSource` instance, the data access layer gets a proxy reference. The proxy intercepts connection acquisition and releasing requests, and, this way, it can monitor its usage.

When using Spring, setting up FlexyPool is fairly easy because the application has total control over the `DataSource` configuration.

In Java EE, database connections should always be fetched from a managed `DataSource`, and one simple way of integrating FlexyPool is to extend the default `DatasourceConnectionProviderImpl` and substitute the original `DataSource` with the `FlexyPoolDataSource`.

[8] https://github.com/vladmihalcea/flexy-pool

For this reason, FlexyPool comes with the following Hibernate connection provider:

```
public class FlexyPoolHibernateConnectionProvider
    extends DatasourceConnectionProviderImpl {

    private transient FlexyPoolDataSource<DataSource> flexyPoolDataSource;

    @Override
    public void configure(Map props) {
        super.configure(props);
        flexyPoolDataSource = new FlexyPoolDataSource<>(getDataSource());
    }

    @Override
    public Connection getConnection() throws SQLException {
        return flexyPoolDataSource.getConnection();
    }

    @Override
    public boolean isUnwrappableAs(Class unwrapType) {
        return super.isUnwrappableAs(unwrapType) ||
            getClass().isAssignableFrom(unwrapType);
    }

    @Override
    public void stop() {
        flexyPoolDataSource.stop();
        super.stop();
    }
}
```

To use the `FlexyPoolHibernateConnectionProvider`, the application must configure the `hibernate.connection.provider_class` property:

```
<property
 name="hibernate.connection.provider_class"
 value="com.vladmihalcea.flexypool.adaptor.FlexyPoolHibernateConnectionProvider"
/>
```

8.3.1 Hibernate statistics

Hibernate has a built-in statistics collector which gathers notifications related to database connections, `Session` transactions and even second-level caching usage. The `StatisticsImplementor` interface defines the contract for intercepting various Hibernate internal events:

```
StatisticsImplementor
openSession()                                void
closeSession()                               void
flush()                                      void
connect()                                    void
prepareStatement()                           void
closeStatement()                             void
endTransaction(boolean)                      void
loadEntity(String)                           void
fetchEntity(String)                          void
updateEntity(String)                         void
insertEntity(String)                         void
deleteEntity(String)                         void
optimisticFailure(String)                    void
loadCollection(String)                       void
fetchCollection(String)                      void
updateCollection(String)                     void
recreateCollection(String)                   void
removeCollection(String)                     void
secondLevelCachePut(String)                  void
secondLevelCacheHit(String)                  void
secondLevelCacheMiss(String)                 void
naturalIdCachePut(String)                    void
naturalIdCacheHit(String)                    void
naturalIdCacheMiss(String)                   void
naturalIdQueryExecuted(String, long)         void
queryCachePut(String, String)                void
queryCacheHit(String, String)                void
queryCacheMiss(String, String)               void
queryExecuted(String, int, long)             void
updateTimestampsCacheHit()                   void
updateTimestampsCacheMiss()                  void
updateTimestampsCachePut()                   void
```

Figure 8.3: Hibernate `StatisticsImplementor` interface

There is a great variety of metrics Hibernate can collect on user's behalf, but, for performance reasons, the statistics mechanism is disabled by default.

To enable the statistics gathering mechanism, the following property must be configured first:

```
<property name="hibernate.generate_statistics" value="true"/>
```

Once statistics are being collected, in order to print them into the current application log, the following logger configuration must be set up:

```
<logger
    name="org.hibernate.engine.internal.StatisticalLoggingSessionEventListener"
    level="info" />
```

With these two settings in place, whenever a Hibernate `Session` (Persistence Context) ends, the following report is displayed in the currently running log.

```
37125102 nanoseconds spent acquiring 10000 JDBC connections;
25521714 nanoseconds spent releasing 10000 JDBC connections;
95242323 nanoseconds spent preparing 10000 JDBC statements;
923615040 nanoseconds spent executing 10000 JDBC statements;
```

The default statistics collector just counts the number of times a certain callback method was called, and, if that is not satisfactory, the application developer can supply its own custom `StatisticsImplementor` implementation.

Dropwizard Metrics

In a high-throughput transaction system, the amount of metric data needed to be recorded can be overwhelming, so storing all these values into memory is not practical at all.

To reduce the memory footprint, the Dropwizard Metrics[a] project uses various *reservoir sampling*[b] strategies, which either employ a fixed-size sampler or a time-based sampling window.

Not only it supports a great variety of metrics (e.g. timers, histograms, gauges), but Dropwizard Metrics can use multiple reporting channels as well (e.g. SLF4J, JMX, Ganglia, Graphite).

For all these reasons, it is better to use a mature framework such as Dropwizard Metrics instead of building a custom implementation from scratch.

[a] https://github.com/dropwizard/metrics
[b] https://en.wikipedia.org/wiki/Reservoir_sampling

8.3.1.1 Customizing statistics

Although the built-in metrics are rather informative, Hibernate is not limited to the default statistics collector mechanism which can be completely customized.

In the upcoming example, the statistics collector also provides the following metrics:

- the distribution of physical transaction time (the interval between the moment a connection is first acquired and when it gets released)
- a histogram of the number of connections acquisition requests during the lifespan of any given transaction (due to the `after_statement` release mode).

The `StatisticsReport` class provides metric storage and report generation features on top of Dropwizard Metrics:

```
public class StatisticsReport {

    private final Logger LOGGER = LoggerFactory.getLogger(getClass());

    private MetricRegistry metricRegistry = new MetricRegistry();

    private Histogram connectionCountHistogram = metricRegistry.
        histogram("connectionCountHistogram");

    private Timer transactionTimer = metricRegistry.
        timer("transactionTimer");

    private Slf4jReporter logReporter = Slf4jReporter
            .forRegistry(metricRegistry)
            .outputTo(LOGGER)
            .build();

    public void transactionTime(long nanos) {
        transactionTimer.update(nanos, TimeUnit.NANOSECONDS);
    }

    public void connectionsCount(long count) {
        connectionCountHistogram.update(count);
    }

    public void generate() {
        logReporter.report();
    }
}
```